

extern "C", Name Mangling, and Linkage

1. What Is Name Mangling?

C++ transforms function names into complex internal symbols to support overloading.

```
// You write:
void print(int x);
void print(double x);
void print(string x);

// C++ compiler generates (MSVC):
// ?print@@YAXH@Z      (print with int)
// ?print@@YAXN@Z      (print with double)
// ?print@@YAXV?$basic...@Z (print with string)

// C compiler does NOT mangle:
void print(int x);    -> _print or print
```

Why C++ mangles:

- To support function overloading
- Each overload needs a unique symbol in the binary
- Mangling encodes: name + params + namespace + class

2. What extern "C" Does

Tells the C++ compiler: **"Compile this function using C naming rules - NO mangling."**

```
// Without extern "C" - mangled
void hello() { }
// symbol: ?hello@@YAXXZ

// With extern "C" - C-style name
extern "C" void hello() { }
// symbol: hello
```

3. Four Major Uses (Not Just Name Mangling)

Use 1 — Calling C Libraries from C++ (Most Common)

```
// math.h is a C library compiled as: _sin, _cos
// extern "C" lets C++ find these symbols
extern "C" {
    double sin(double x);
    double cos(double x);
}

// This is what system headers do internally:
extern "C" {
    int printf(const char* fmt, ...);
    void* malloc(size_t size);
    char* strcpy(char* dst, const char* src);
}
```

Use 2 — Building DLLs for Other Languages

```
// mylib.cpp
extern "C" {
    __declspec(dllexport) int addNumbers(int a, int b) {
        return a + b;
    }
}
```

```
# Python calling your C++ DLL
import ctypes
lib = ctypes.CDLL("mylib.dll")
result = lib.addNumbers(3, 4) # works - clean symbol name
```

Use 3 — Mixing C and C++ Files

```
// engine.c - pure C
void engineInit() { printf("Engine started\n"); }
```

```
// game.cpp - C++ calling C
extern "C" {
    void engineInit();
}
int main() {
    engineInit(); // links correctly
}
```

Use 4 — Plugin Systems / Dynamic Loading

```
// plugin.cpp
extern "C" {
    void onLoad() { cout << "Loaded"; }
    void onUpdate() { cout << "Update"; }
}

// host.cpp - load at runtime
auto onLoad = (void(*)())dlsym(handle, "onLoad");
onLoad();
```

4. Standard Header Pattern

```
// mylib.h - usable from BOTH C and C++

#ifndef MYLIB_H
#define MYLIB_H

#ifdef __cplusplus
extern "C" {
#endif

    void myInit();
    int myAdd(int a, int b);
    void myCleanup();

#ifdef __cplusplus
}
#endif

#endif
```

5. What `extern "C"` Does NOT Allow

```
extern "C" {
    void foo(int x = 5);           // ERROR - no default arguments
    void foo(double x);           // ERROR - no overloading
    template<typename T> void f(); // ERROR - no templates
    class Car { };                // ERROR - no classes
}

// But the IMPLEMENTATION can use C++ freely:
extern "C" void foo(int x) {
    vector<int> v;                // OK - C++ inside
    string s = "hi";              // OK
}
```

```
    Car car;           // OK
}
```

6. Internal vs External Linkage

External linkage -> symbol visible to linker from other .cpp files
Internal linkage -> symbol exists ONLY in this .cpp file

```
// file_a.cpp
void publicFunc() {}           // external linkage

namespace {
    void privateFunc() {}      // internal linkage
}

static int privateInt = 5;     // internal linkage

// file_b.cpp
void publicFunc();             // OK - found
void privateFunc();            // LINKER ERROR
extern int privateInt;          // LINKER ERROR
```

7. `extern` Alone (Different Thing!)

```
// extern for variables - "defined elsewhere"
// file1.cpp
int globalScore = 0;           // definition

// file2.cpp
extern int globalScore;         // declaration - "exists in another file"
globalScore = 100;              // OK
```

```
extern "C"    void foo();       // C linkage    - no mangling
extern "C++"  void bar();       // C++ linkage - default, rarely written
extern       int x;              // variable: defined elsewhere
```

8. Common Error — Declaration/Definition Mismatch

```
// header.h
extern "C" void PrintA(int a);    // declaration with C linkage

// source.cpp
void PrintA(int a) {}            // definition WITHOUT extern "C"
// -> different mangled name -> LINKER ERROR LNK2019/LNK1120
```

Fix: declaration and definition must agree on `extern "C"`.

9. Symbol Resolution — Object Files vs Libraries

```
LINKER SEARCH ORDER:
1. FIRST -> your .obj files (always linked, fully)
2. THEN  -> libraries (.lib) - only objects needed to resolve missing symbols
```

This explains why you can "override" library functions:

```
// myFile.cpp
extern "C" double sqrt(double x) {
    return x + x;    // fake sqrt
}

// main.cpp
cout << sqrt(2.3);  // calls YOUR sqrt (returns 4.6), not math.h's
```

Why? Your `sqrt` is in `myFile.obj` (always linked). The linker resolves `sqrt` from your obj first and never reaches the CRT library's `sqrt`.

Caveat: In Release mode (`/O2`), `sqrt` becomes a compiler **intrinsic** (built into the compiler) and you cannot redefine it → error C2169.

10. Summary

```
extern "C" solves:
- Calling C libraries from C++ (stdio, math, OpenGL, SQLite)
- Building DLLs/SOs for Python/C#/Java/Unity
- Mixing C and C++ in one project
- Plugin systems / dynamic loading
```

Root cause it addresses:

- C++ name mangling makes symbols unpredictable
- `extern "C"` = use C naming = predictable symbols
- The bridge between C++ and the rest of the software world

extern "C" does NOT allow:

- Overloading, default args, templates, classes inside it
- (But the function body can use any C++ features)